Mini Proyecto 01 - Ballistic Flight Simulator v1.0

Python Engineering - Consolidación de lógica, loops, validación y Euler explícito

Este mini proyecto cierra el primer bloque del curso. El objetivo no es añadir mucha sintaxis nueva, sino integrar en un solo programa todo lo aprendido: variables, validación, condicionales, loops, estado, acumuladores, Euler explícito y validación frente a una solución analítica.

1. Objetivo del proyecto

• Construir un simulador vertical 1D con gravedad constante y sin resistencia aerodinámica.

• Propagar el estado **[t, h, v]** mediante Euler explícito con paso temporal fijo **dt**.

• Detectar el apogeo aproximado y el primer paso que cruza el suelo.

• Comparar tiempo de vuelo, altura máxima y velocidad final con la solución analítica.

• Estudiar cómo cambia el error cuando se reduce **dt**.

• Entregar un script claro, documentado y reproducible que pueda evolucionar a versiones posteriores.

2. Modelo físico

Usaremos el mismo modelo del EX08. El eje vertical positivo apunta hacia arriba y **g > 0** representa la magnitud de la gravedad.

$$dh/dt = v$$

$$dv/dt = -g$$

$$h_{(k+1)} = h_k + v_k\, dt$$

$$v_{(k+1)} = v_k - g\, dt$$

$$t_{(k+1)} = t_k + dt$$

Idea clave: calcular primero todo el nuevo estado con valores del estado k y actualizar después. Así se mantiene Euler explícito.

3. Solución analítica de referencia

La solución analítica no genera la trayectoria numérica; solo sirve como referencia independiente para validar el simulador.

$$h_{max} = h_0 + v_0^2 / (2g)$$

$$t_f = (v_0 + \sqrt{v_0^2 + 2\, g\, h_0}) / g$$

$$v_f = -\sqrt{v_0^2 + 2\, g\, h_0}$$

4. Arquitectura lógica

```
INPUTS
  h0, v0, t0, g, dt
      |
VALIDATION
      |
INITIAL STATE
  t=t0, h=h0, v=v0
      |
WHILE h >= 0
  1. compute h_new, v_new, t_new
  2. update max_h
  3. replace old state
  4. count iteration
      |
FINAL NUMERICAL RESULTS
      |
ANALYTICAL REFERENCE
      |
ERRORS + SUMMARY
```

5. Requisitos mínimos

Bloque	Requisito
Entradas	h0 >= 0, v0 > 0, g > 0, dt > 0; t0 puede ser una referencia temporal.
Integrador	Euler explícito. No usar SciPy, RK4 ni la fórmula cerrada para propagar la trayectoria.
Estado	Usar variables separadas para estado actual y estado nuevo.
Máximo	Guardar max_h comparando el valor actual con el máximo acumulado.
Parada	Finalizar al detectar el primer estado con h < 0.
Validación	Calcular referencias analíticas y errores porcentuales.
Salida	Mostrar resumen final legible con dt, iteraciones, h_max, t_f y v_f.

6. Corrección importante para el error de velocidad

> **Como la velocidad analítica de impacto es negativa, el denominador del error relativo debe usar su valor absoluto: abs(analytical_v_f). Si divides por analytical_v_f directamente, el porcentaje puede salir negativo aunque estés midiendo una magnitud de error.**

error_v_f = abs(analytical_v_f - v_f) / abs(analytical_v_f) * 100

7. Experimento de convergencia

Ejecuta el mismo simulador sin cambiar ninguna condición física y modifica solo **dt**. Registra los resultados en una tabla.

dt [s]	h_max Euler [m]	Error h_max [%]	t_f Euler [s]	Error t_f [%]
1.0				
0.5				
0.1				
0.05				

dt [s]	h_max Euler [m]	Error h_max [%]	t_f Euler [s]	Error t_f [%]
0.01				

Pregunta principal: ¿qué observas cuando dt disminuye? No busques solo que el error sea pequeño: observa la tendencia.

8. Entregables del mini proyecto

```
Python_Engineering/
■■■ 01_python_bootcamp/
    ■■■ week_01_logic/
        ■■■ class_01_equation_to_algorithm/
            ■■■ mini_project/
                ■■■ ballistic_flight_simulator_v1/
                    ■■■ ballistic_flight_simulator_v1.py
                    ■■■ README.md
                    ■■■ notes/
```

• **ballistic_flight_simulator_v1.py**: el programa final.

• **README.md**: breve descripción del modelo, inputs, outputs, ecuaciones y limitaciones.

• **notes/**: los PDFs del EX08/Euler y este briefing del mini proyecto.

9. Casos de prueba

Caso	h0	v0	g	dt	Qué comprobar
Base	0	50	9.81	0.1	Ejecución nominal y validación analítica.
Paso grande	0	50	9.81	1.0	Error de Euler claramente visible.
Paso pequeño	0	50	9.81	0.01	Mayor proximidad a la referencia.
Altura inicial	100	50	9.81	0.1	Usar fórmulas analíticas generales.
Inválido	0	50	9.81	0	El programa debe detenerse antes del while.

10. Qué NO hacer todavía

• No usar **solve_ivp**, RK2 o RK4.

• No convertir aún el proyecto a clases.

• No añadir drag, atmósfera, masa variable ni thrust.

• No obsesionarse todavía con detectar el instante exacto de impacto; el cruce entre pasos se estudiará después.

11. Bonus opcional

> **Si quieres añadir una sola mejora, calcula también la energía mecánica aproximada E = g h + v^2/2 por unidad de masa en cada paso y observa si Euler la conserva exactamente. No es requisito para cerrar el proyecto.**

12. Criterios de cierre

Criterio	Completado
Inputs y validaciones coherentes	■
Euler explícito correctamente implementado	■

Criterio	Completado
max_h, t_f y v_f calculados	■
Referencias analíticas correctas	■
Errores porcentuales correctos	■
Estudio con varios dt realizado	■
Código y README legibles	■

13. Learning Tracker

Concepto	Estado
Condicionales y validación	Consolidado
for / while	Consolidado
Estado dinámico k -> k+1	Consolidado
Euler explícito	Aplicado
Error de discretización	Aplicado
Validación analítica	Aplicado
Convergencia al reducir dt	A estudiar en el proyecto
Diseño incremental de simuladores	Introducido

Objetivo final: que puedas explicar por qué el simulador funciona, de dónde procede su error y cómo comprobar que mejora al reducir dt.